

RecruitAI

Ranking candidates the way a great recruiter would

A hybrid scoring engine that reads career history, skills, behavior, and platform activity together — not just keywords.

The problem

Keyword filters miss the candidates who actually fit

Recruiters drown in volume

Hundreds of profiles per role. No time to read every career history line by line.

Keyword search is brittle

"PyTorch" ≠ "fine-tuning LLMs" ≠ "production ML" to a string match, even though a recruiter reads them as the same signal.

Fit is multi-dimensional

Skills matter, but so does career trajectory, responsiveness, and whether the person is even reachable right now.

The approach

A hybrid scoring engine, not a single model

Pure embeddings miss structured facts (years of experience, response rate). Pure rules miss semantic nuance. RecruitAI combines both into one explainable score.

1

Parse the JD

Extract the implied experience range and requirements/skills section with lightweight, zero-cost NLP parsing.

2

Score every candidate

Six independent signals computed from career history, skills, education, behavior, and platform activity.

3

Rank & explain

Weighted composite score, sorted shortlist, with a plain-English reason per candidate.

How fit is scored

Six signals, weighted into one score

Semantic relevance	32%	TF-IDF cosine similarity across headline, summaries, job descriptions, skills — captures phrases like "fine-tuning LLMs", not isolated words.
Skill match	28%	Explicit skill list vs JD requirements, weighted by proficiency and endorsements.
Experience fit	14%	Years of experience vs the range parsed from the JD, with a forgiving curve.
Behavioral reliability	12%	Recruiter response rate, interview completion, offer acceptance, profile completeness — will they actually respond?
Career stability	8%	Average tenure & job count — flags job-hopping risk or stagnation.
Education	6%	Institution tier as a light tiebreaker signal.

Design decisions

Why TF-IDF, not a downloaded embedding model

- No GPU or paid API required — runs fully offline, zero marginal cost per query
- Builds a 100,000-candidate index in under a minute on a single CPU
- Bigram TF-IDF still captures meaningful phrases ("image classification", "fine-tuning LLMs")
- Sub-second ranking once the index is cached — recruiters get instant results
- Combined with structured scoring, so it doesn't have to carry semantic understanding alone
- Swappable: scoring.py exposes clean seams to drop in an LLM-based JD parser or embedding model later

System architecture

Backend + lightweight UI, nothing to babysit



Every request is stateless and explainable: same JD + same candidate pool always reproduces the same ranked shortlist with a visible score breakdown.

What recruiters get

A shortlist they can trust — and verify

#1 Ira Mukherjee

58

NLP Engineer · Aganitha · Vizag, AP · 6.6 yrs · OPEN TO WORK

Matched skills: PyTorch · LLMs · Python · Fine-tuning LLMs

"Strong overlap on PyTorch, LLMs, Python, Fine-tuning LLMs. 6.6 yrs experience matches the role's 4-7 yr target. Actively open to work. Historically responsive to recruiter outreach. Stable career history, low job-hopping risk."



Recruiters see the score, the evidence, AND the breakdown — never a black-box number.

Exportable as CSV for ATS import. Same ranking is reproducible from the API for audit purposes.

What's next

Where this goes with more time

- **LLM-based JD parsing**
Replace the regex parser with an LLM call (e.g. Claude) for higher-precision requirement extraction on messy JDs.
- **Learned weights**
Once recruiters label outcomes (good hire / no response / rejected), fit the six signal weights with logistic regression instead of hand-tuning.
- **Dense embeddings**
Add a sentence-embedding model alongside TF-IDF for deeper semantic matches once GPU/API budget allows.
- **Recruiter feedback loop**
Let recruiters thumbs up/down a shortlist entry — feed that signal back into ranking for that role.